Symbols, finite words, and the empty word

An alphabet Σ is a finite nonempty set of symbols. A symbol is one indivisible token, even if its printed label has several characters. A word $w = a_1 \cdots a_n$ is a finite sequence of length n. The empty word ε has length zero and differs from a blank-space symbol.

Define $\Sigma^0 = \{\varepsilon\}$, Σ^n as all length-n words, $\Sigma^* = \bigcup_{n \geq 0} \Sigma^n$, and $\Sigma^+ = \bigcup_{n \geq 1} \Sigma^n$. Thus $\Sigma^* \setminus \Sigma^+ = \{\varepsilon\}$, with $\varnothing \neq \{\varepsilon\}$.

For $|\Sigma| = s$, independent position choices give $|\Sigma^n| = s^n$. Listing by length and a fixed within-length symbol order enumerates all words, so Σ^* is countably infinite. Each word being finite is compatible with infinitely many words.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Concatenation, prefixes, and reversal

Concatenation places sequences in order, with $|uv| = |u| + |v|$, identity ε, and associativity. It generally does not commute, as $01 \neq 10$. A common prefix cancels: deleting its $|x|$ positions in $xu = xv$ yields $u = v$.

A prefix satisfies $w = uv$, a suffix $w = vu$, and a substring $w = xuy$. Substrings are contiguous; subsequences only preserve position order. Thus 01 is not a prefix of 001, but is a suffix, substring, and subsequence.

Define $\varepsilon^R = \varepsilon$, $(aw)^R = w^R a$. Induct on $|u|$ to prove $(uv)^R = v^R u^R$. The empty case holds. Writing $u = ax$, the step gives $((ax)v)^R = (xv)^R a = v^R x^R a = v^R u^R$. Reversing positions twice also gives $(w^R)^R = w$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Language concatenation need not multiply cardinalities

A language is any subset $L \subseteq \Sigma^*$, empty, finite, or infinite. Define $AB = \{uv : u \in A, v \in B\}$. The concatenation map $A \times B \to AB$ is onto, but a word may have different split points. Thus finite languages only guarantee $|AB| \leq |A||B|$.

For $A = \{a, aa\}, B = \{\varepsilon, a\}$, the four pairs produce a, aa, aa, aaa, only three words. The duplicate is a resulting word, not a pair.

The language identity is $\{\varepsilon\}$, so $A\{\varepsilon\} = \{\varepsilon\}A = A$. In contrast $A\varnothing = \varnothing A = \varnothing$. Concatenation generally does not commute: singleton languages produce ab versus ba.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Language equalities and one-way inclusions

Both $(AB)C$ and $A(BC)$ contain exactly words abc with factors in their respective languages, proving associativity. A suffix in $B \cup C$ belongs to one of those languages, giving $A(B \cup C) = AB \cup AC$; right distribution follows similarly.

Only $A(B \cap C) \subseteq AB \cap AC$ is automatic. A shared decomposition suffices on the left, while the right may use two decompositions. With $A = \{\varepsilon, a\}, B = \{\varepsilon\}, C = \{a\}$, the left is empty and the right $\{a\}$.

Inclusion $A \subseteq B$ gives $A^n \subseteq B^n$, since every factor remains permitted. Inductively the zero powers both equal $\{\varepsilon\}$, and concatenation preserves inclusion at the next step.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Kleene star counts factors, not character length

Set $L^0 = \{\varepsilon\}$, $L^{n+1} = LL^n$, $L^* = \bigcup_{n \geq 0} L^n$, and $L^+ = \bigcup_{n \geq 1} L^n$. The exponent counts language factors, whose lengths need not be one.

We have $\varnothing^* = \{\varepsilon\}$, $\varnothing^+ = \varnothing$, and $\{\varepsilon\}^* = \{\varepsilon\}^+ = \{\varepsilon\}$. For general L, its positive closure may contain the empty word. The alphabet's Σ^+ excludes it because each symbol has positive length.

The star contains ε and L, and merging finite factor lists proves closure under concatenation. Conversely, induction on factor count shows every concatenation-closed set containing ε, L contains every L^n, hence L^*. This proves its minimality.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Star identities by splitting and regrouping

A nonempty factor list can be split before its first or after its last factor, proving $LL^* = L^*L = L^+$, even with empty-word factors. Merging two lists proves $L^*L^* \subseteq L^*$; concatenating with ε proves the reverse. Flattening finite lists of finite lists similarly gives $(L^*)^* = L^*$, $(L^*)^+ = L^*$, $(L^+)^* = L^*$.

$$(A \cup B)^* = (A^* \cup B^*)^* = (A^*B^*)^*.$$

For the first equality, flatten each starred factor into factors from $A \cup B$; conversely every individual factor belongs to one star. For the second, expand each A^*B^* factor into two blocks. Conversely write an A^* factor as $u\varepsilon$, or a B^* factor as εv. Both inclusions need justification; stars do not cancel by appearance.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Soundness and completeness of recursive languages

Start at word zero and permit adding one on either end, taking only words generated by finitely many applications. Exactly the words with one zero result. Soundness holds because rules preserve the zero count. Every target 1^i01^j is generated by i left additions and j right additions, proving completeness.

Define balanced parentheses by $\varepsilon \in D$ and $u, v \in D \Rightarrow (u)v \in D$. The intended words have nonnegative prefix height and final height zero. Induction on generation proves soundness: enclosing u raises its heights by one, returns to zero, then appends admissible v.

For completeness, induct on length. A nonempty admissible word starts with an opening parenthesis. Its first height-zero return uniquely splits it as $(u)v$, with both shorter pieces admissible, hence generated by induction. Requiring the least generated set excludes arbitrary extra words unsupported by the rules.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Grammar symbols and one derivation

Consider $G = (N, \Sigma, P, S)$, with nonterminals $N = \{S, A, B\}$, terminals $\Sigma = \{a, b\}$, and start symbol S. Its productions P are

$$S \to AB, \qquad A \to aA \mid a, \qquad B \to b.$$

Each step replaces one nonterminal using one production. Apply $A \to aA$ once, then terminate A with $A \to a$, and replace B by b:

$$S \Rightarrow AB \Rightarrow aAB \Rightarrow aaB \Rightarrow aab.$$

The intermediate form aAB still contains nonterminals; aab contains only terminals and is a generated word.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The generated language: both inclusions

The grammar generates exactly

$$L(G) = \{a^k b : k \geq 1\}.$$

Every generated word has this form. The sole start rule gives AB. Each $A \to aA$ adds one a and retains A. A terminal derivation must eventually use $A \to a$ to eliminate A, producing at least one a; B can only become one final b. Thus every generated word is $a^k b$ for $k \geq 1$.

Every target word is generated. For any integer $k \geq 1$, apply $S \to AB$, then $A \to aA$ exactly $k-1$ times, followed by $A \to a$ and $B \to b$. The result is $a^k b$; $k = 2$ gives aab. The empty word ε cannot be generated because every terminating derivation contains at least one a and one b.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A Mealy machine outputs a whole stream

A fixed-initial-state Mealy transducer is $(Q, \Sigma, \Gamma, \delta, \lambda, q_0)$, with finite state, input, and output alphabets, and total maps $\delta : Q \times \Sigma \to Q$, $\lambda : Q \times \Sigma \to \Gamma$. Each input emits one symbol; edges are labeled input/output.

Extend by $\delta^*(q, \varepsilon) = q, \lambda^*(q, \varepsilon) = \varepsilon$, and

$$\delta^*(q, wa) = \delta(\delta^*(q, w), a),$$
$$\lambda^*(q, wa) = \lambda^*(q, w)\lambda(\delta^*(q, w), a).$$

Output uses the state before reading the symbol, then the transition updates it. Substituting the new state into the output function can corrupt every step. Finite states summarize history while permitting arbitrarily long input.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A serial adder remembers only carry

Input binary numbers low bit first, with paired symbols $xy \in \{00, 01, 10, 11\}$. State $c \in \{0, 1\}$ stores carry, initially zero. Let $t = x + y + c$, output $z = t \bmod 2$, and update $c' = \lfloor t/2 \rfloor$.

Old carry	00: new/output	01	10	11
0	0/0	0/1	0/1	1/0
1	0/1	1/0	1/0	1/1

After k pairs, the low-part values satisfy $X_k + Y_k = Z_k + 2^k c_k$. Initially all are zero. Multiply the next-bit identity $x + y + c_k = z + 2c_{k+1}$ by 2^k and add to the previous invariant to preserve it. This proves arbitrary input lengths, rather than only eight one-step cases.

Thirteen plus six and the final carry

Input $13 = (1101)_2$ and $6 = (0110)_2$ low first, followed by one padding 00 to emit the final carry.

Bit k	Pair	Old c	t	Output	New c
0	10	0	1	1	0
1	01	0	1	1	0
2	11	0	2	0	1
3	10	1	2	0	1
4, padding	00	1	1	1	0

Output arrives as 11001; reversing gives conventional 10011, nineteen. Before padding, $13 + 6 = 3 + 16 \cdot 1$; after padding carry is zero and output value nineteen. Ignoring the final state after only four inputs would retain just the low value three.

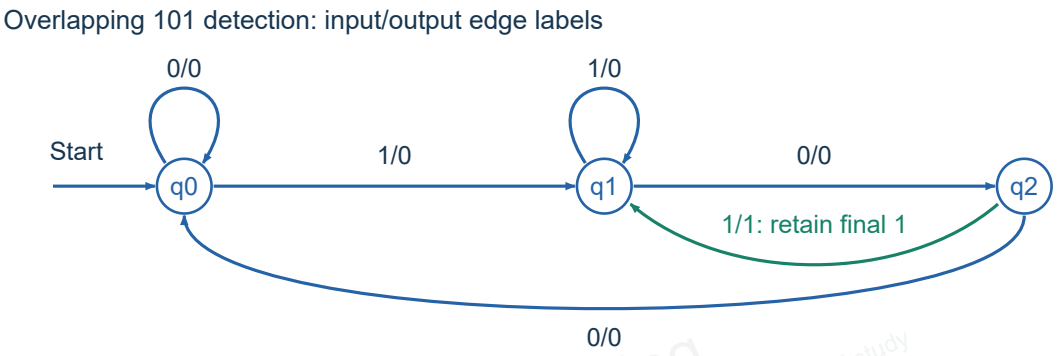
Detecting 101 requires overlap memory

States q_0, q_1, q_2 store the longest suffix matching a proper prefix of 101, respectively $\varepsilon, 1, 10$. Emit one exactly when the latest symbol completes 101.

State	Input 0: new/output	Input 1: new/output
q_0	$q_0/0$	$q_1/0$
q_1	$q_2/0$	$q_1/0$
q_2	$q_0/0$	$q_1/1$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Detecting 101 requires overlap memory (continued)



Reading 10101 gives states $q_0, q_1, q_2, q_1, q_2, q_1$ and output 00101. A completed match returns to q_1, since its final one can start another. Returning to q_0 misses positions three through five. Every table cell preserves the longest matching suffix; induction from empty input proves output one iff the last three input bits are 101.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Position restrictions require a phase

To report matches only at positions divisible by three, also store consumed length modulo three. Use nine product states (q, r), initially $(q_0, 0)$. For input a, let the original table give q', z, set $r' = (r + 1) \bmod 3$, and emit one iff $z = 1 \wedge r' = 0$. The new state is (q', r').

Each component invariant is preserved, proving both conditions. Output for 10101 becomes 00100, and for 101101 it is 001001. Resetting phase only upon matches would measure distance from the previous match, not global position. Nine is a construction upper bound, not an assertion of minimality.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fixed delay, state lower bounds, and output length

A delay of $k \geq 1$ ticks with initial zero padding uses queue state $(b_1, \ldots, b_k) \in \{0, 1\}^k$. On input a, emit old b_1 and shift to $(b_2, \ldots, b_k, a)$, starting at all zeros.

Induction shows the queue stores the latest k inputs, padded on the left when needed. Output t is zero for $t \leq k$, and input $t - k$ afterward. Length-n input gives length-n output, without automatic extra ticks.

For delay two and input 10110, states are $00 \to 01 \to 10 \to 01 \to 11 \to 10$, with output 00101. Two explicit padding inputs are needed to flush the remaining bits.

At least 2^k states are necessary. After any of the distinct length-k histories, append common 0^k. The next output block reproduces the history, so different histories cannot have the same machine state.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

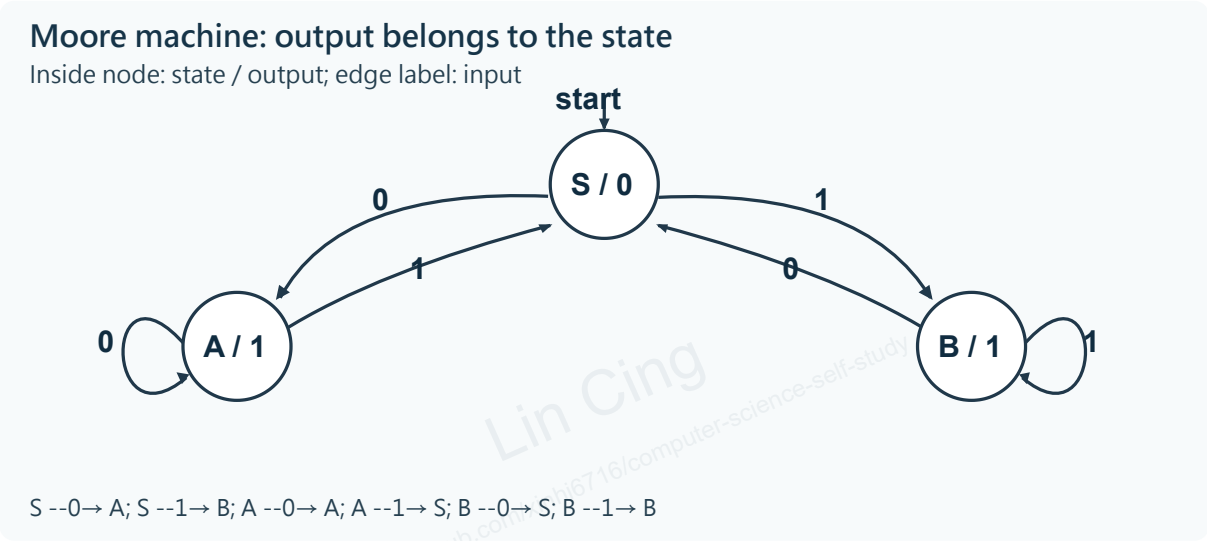
Align Mealy and Moore output timing

A Moore machine's output $h : Q \to \Gamma$ depends only on state. Here each input first transitions, then emits the new state's output; initial output is excluded. Including initial output or emitting before transition introduces a one-symbol shift.

From Moore to Mealy, keep transitions and set $\lambda(q, a) = h(\delta(q, a))$. From Mealy to Moore, states $(q, z) \in Q \times \Gamma$ record the new state and last output; let $h(q, z) = z$, and transition to $(\delta(q, a), \lambda(q, a))$. Add a fresh initial s, choose any $z_0 \in \Gamma$, and set $h(s) = z_0$, keeping the Moore output function total. On its first input, s goes to $(\delta(q_0, a), \lambda(q_0, a))$; under this timing convention, $h(s)$ is not counted in the output stream.

Induction on input length identifies the first component with the original state and second with its last output. Entire streams agree for nonempty input, and both give the empty output on empty input. Finiteness is preserved, though states may increase.

Moore machine: states and outputs



The value inside each node is the state output h; edge labels are inputs. The initial state is S, with $h(S) = 0$ and $h(A) = h(B) = 1$. Every state has exactly one transition for each input.

One path under two output conventions

Read 0011 left to right. For each input, take the transition from the current state, then read the output of the new state.

Time	Initial	Read 0	Read 0	Read 1	Read 1
State	S	A	A	S	B
State output	0	1	1	0	1

Including the initial output gives **01101**, one symbol longer than the input. This chapter emits after each transition and excludes the initial output, giving **1101**, the same length as the input. Fix the convention before comparing Moore and Mealy machines.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Reachable, transient, sink, and submachine

State q is reachable from p if $\delta^*(p, w) = q$ for some $w \in \Sigma^*$, including self-reachability through empty input. This chapter uses that convention; the reference textbook restricts reachability here to distinct states connected by a nonempty transfer string. A state is transient if no nonempty word returns to it. Absence of a self-loop alone is insufficient; a two-edge cycle also returns.

A sink satisfies $\delta(q, a) = q$ for all symbols, hence remains there on all words by induction. A submachine's selected states must be closed under its retained input alphabet; a circled region with outward transitions does not qualify.

Strong connectivity means mutual reachability of every state pair. These transition properties do not determine outputs. A sink of an acceptor may accept or reject according to its membership in the accepting set.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

BFS proves shortest transfer sequences

A transfer sequence is a shortest word from one specified state to another. With unit symbol costs, BFS discovers states by word length. Store a predecessor and input label at first discovery, then backtrack for the word.

State	0	1
S	A	B
A	A	T
B	T	B
T	T	T

BFS from S finds S at distance 0, A and B at distance 1, then T at distance 2. Both 01 and 10 are shortest.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

BFS shortestness and cost

A revisited state need not be expanded with a longer prefix: the same state followed by the same suffix gives the same result, so the longer prefix cannot yield a shorter path.

A shortest path between distinct reachable states repeats no vertex; deleting a repeated-vertex cycle would shorten it. Its length is at most $|Q| - 1$. Searching the complete transition table costs $O(|Q||\Sigma|)$ time and $O(|Q|)$ search space. Unequal symbol costs invalidate the unit-cost BFS shortestness argument.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Acceptors differ from stream transducers

A DFA is $M = (Q, \Sigma, \delta, q_0, F)$, with $q_0 \in Q$, $F \subseteq Q$, and total δ. It accepts w iff $\delta^*(q_0, w) \in F$, defining $L(M)$.

Each input has one complete path, and acceptance is decided at its end. Passing an accepting state midway does not suffice. The empty word is accepted iff $q_0 \in F$. A double circle marks acceptance; a source-free arrow marks the initial state.

Mealy output is emitted at every step and has no built-in accepting set. Using final output one as acceptance requires an additional empty-input convention, since empty input emits nothing.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The extended-transition concatenation lemma

With $\delta^*(q, \varepsilon) = q$ and $\delta^*(q, wa) = \delta(\delta^*(q, w), a)$,

$$\delta^*(q, uv) = \delta^*(\delta^*(q, u), v).$$

Induct on $|v|$. The empty suffix is immediate. If $v = xa$, use the induction hypothesis for x, then append the final transition on a. This proves that a common suffix from equal states gives equal results.

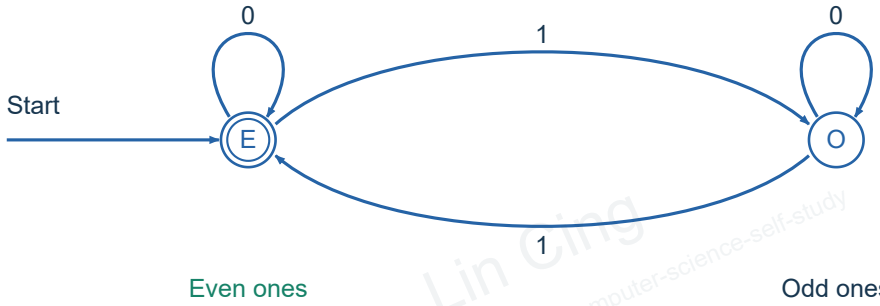
A machine design needs a state invariant proved from empty input through every symbol, then acceptance iff the target property. Finite tests may refute a design but cannot replace this universal proof.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A two-state acceptor for even ones

States E,O record even and odd counts of ones. Start E and accept only E. Zero leaves state unchanged; one toggles it.

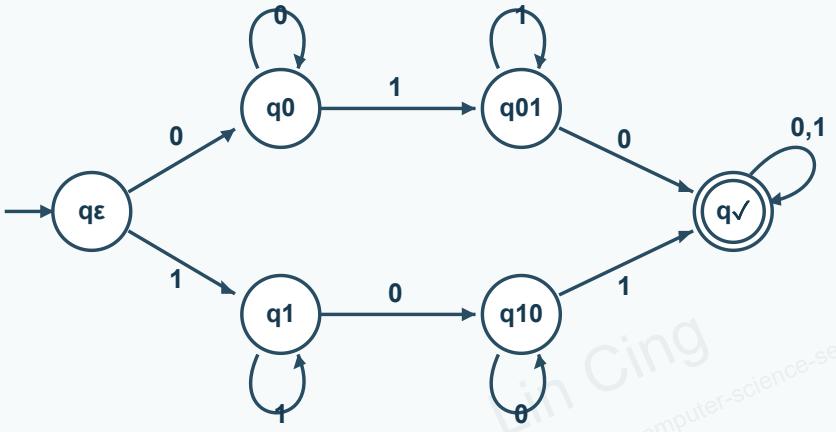
Even ones: E is initial and accepting; edges show inputs



Empty input has zero ones. Appending zero preserves parity, while one flips it, matching all four transitions. Induction proves the state meanings and the desired acceptance equivalence.

The path for 1010 is $E \to O \to O \to E \to E$, accepting. Word 1011 ends at O and rejects, regardless of earlier visits to E.

A DFA for both 01 and 10



Double circle: the only accepting state. Each state has both 0 and 1 transitions.

States record which adjacent changes have occurred; only $q_{\checkmark}$ accepts. Acceptance is decided after the whole word is read, and seeing just one pattern is insufficient.

Trace both patterns

Input	Path	Result
010	$q_\varepsilon \to q_0 \to q_{01} \to q_\checkmark$	Accept
101	$q_\varepsilon \to q_1 \to q_{10} \to q_\checkmark$	Accept
01	$q_\varepsilon \to q_0 \to q_{01}$	Reject
10	$q_\varepsilon \to q_1 \to q_{10}$	Reject

The empty word stays at q_ε, so it is rejected. By induction on input length, q_{01} records that 01 has occurred and 10 has not; the last bit must then be 1, so reading 0 creates the first 10. The case q_{10} is symmetric. Once reached, $q_\checkmark$ retains both patterns, so acceptance is equivalent to having both.

A modulo-three summary of binary values

Allow leading zeros and assign value zero to empty input. Appending bit b changes value v to $2v + b$, so remember only its residue:

r	0	1
0	0	1
1	2	0
2	1	2

Start at residue zero and accept zero. The initial residue and preservation under multiplication by two plus a bit prove the invariant by induction. Requiring nonempty representations instead needs an additional seen-input flag; merely changing the verbal specification would leave the old machine accepting empty input.

Product machines preserve two invariants

For two total DFAs on one alphabet, use states $Q_1 \times Q_2$, paired initial state, and componentwise updates. Induction gives after w exactly the pair of original final states.

Accepting pairs in $F_1 \times F_2$ gives intersection. Accepting $(F_1 \times Q_2) \cup (Q_1 \times F_2)$ gives union. First accepting and second rejecting gives difference; exactly one accepting gives symmetric difference. Substituting the paired-state invariant proves every language identity.

There are at most $|Q_1||Q_2|$ states; discard unreachable pairs. Both acceptors read the same input. This is not feeding one whole-word output into another machine.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A six-state product for multiples ending in one

Store (r, ℓ), residue modulo three and whether the latest bit is one, using empty-input flag $\ell = 0$. Start $(0, 0)$, update on bit b to $((2r + b) \bmod 3, b)$, and accept only $(0, 1)$.

State	0	1
(0,0)	(0,0)	(1,1)
(0,1)	(0,0)	(1,1)
(1,0)	(2,0)	(0,1)
(1,1)	(2,0)	(0,1)
(2,0)	(1,0)	(2,1)
(2,1)	(1,0)	(2,1)

Word 10101 follows $(0, 0) \to (1, 1) \to (2, 0) \to (2, 1) \to (1, 0) \to (0, 1)$, accepting value 21. Word 1010 has value ten and ends in zero, finishing $(1, 0)$ and rejecting. Empty input rejects too because it has no final one.

Complete transitions before complementing

Every word in a total DFA reaches one final state. Replacing F by $Q \setminus F$ therefore complements its language relative to fixed Σ^*.

For partial transitions with missing edges rejecting, flipping circles fails. A sole initial accepting state s on alphabet $\{a\}$, with no edges, accepts only ε. Flipping it yields the empty language instead of the correct a^+.

First add a rejecting sink d, direct every missing edge to it, and loop it on all symbols. This preserves the language. Then flip acceptance, making d accepting and s rejecting, to obtain a^+. Missing-edge rejection is a definite semantics, not a third status between acceptance and rejection.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

NFA acceptance means one complete path exists

An epsilon-NFA can use $\Delta : Q \times (\Sigma \cup \{\varepsilon\}) \to \mathcal{P}(Q)$. Symbols may have zero or multiple successors; epsilon edges consume no input. Acceptance requires a finite path starting at the initial state, whose labels after removing epsilons equal the whole input, and ending accepting.

Not every branch needs success. Reaching an accepting state does not excuse unread input. No available branch means rejection; epsilon cycles do not require infinite execution.

Let $E(S)$ be states reachable from S by zero or more epsilon edges. A work queue with visited states adds each new state once, processing at most $|Q|$ states even with cycles. Induction on reachable-path length proves completeness.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Subset construction and its general proof

The deterministic start is $S_0 = E(\{q_0\})$, with subsets as states and update

$$T(S, a) = E\left(\bigcup_{q \in S} \Delta(q, a)\right).$$

A subset accepts iff it intersects the NFA accepting set. If the empty subset is reachable, retain it as a rejecting sink: every input leaves it empty. If constructing only reachable subsets and the empty subset is unreachable, it need not be created.

Induct on input length. At zero, the subset is precisely epsilon-reachable states. Appending a first takes an a-edge from a currently reachable state, then any epsilon edges, exactly the update. Conversely every updated point has such a path. Thus final acceptance is equivalent to existence of an NFA accepting path.

At most $2^{|Q|}$ subsets occur; construct only reachable ones. NFAs recognize no additional language class, though they may provide smaller descriptions.

An epsilon-NFA for suffix ten

Start s has an epsilon edge to p. State p loops on zero and one and also has a one-edge to q. State q reads zero to accepting r. All unlisted transitions are empty. The machine guesses the start of the last two bits and accepts exactly suffix 10.

NFA for suffix 10: accept if a complete input path ends at r



Initial s takes an epsilon edge; p retains every possible prefix.

Unlisted transitions are empty; only r accepts.

Subset	0	1	Accepting?
S={s,p}	P={p}	Q={p,q}	No
P={p}	P	Q	No
Q={p,q}	R={p,r}	Q	No
R={p,r}	P	Q	Yes

An epsilon-NFA for suffix ten (continued)

Word 1010 follows $S \to Q \to R \to Q \to R$, accepting. After R, reading one loses the old accepting r branch, but p retains a fresh potential match start. Word 0101 ends Q and rejects.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

NFA constructions for union and concatenation

First rename component states disjointly. For union, a fresh initial state epsilon-branches to both initial states; accept their combined accepting set. Every accepting path chooses one component, and any component accepting path extends from the new start, proving union in both directions.

For concatenation, retain the first start and add epsilon edges from each first-machine accepting state to the second start. Accept only second-machine accepting states. Split an accepting path at its cross-machine edge into uv, accepted by the respective components. Conversely concatenate such paths. Empty factors are handled by epsilon paths.

Disjoint renaming prevents accidental returns from the second machine to the first. Merging same-named states or retaining first-component accepting states as final can introduce erroneous acceptance after only the first factor.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Explicit path proofs for star and reversal

For star, add start s and accepting t, with epsilon edges s to t and the old start, and from every old accepting state to t and the old start. Only t accepts. The direct s-to-t path represents zero factors. Every restart begins another original-language factor; conversely concatenate accepting paths for any finite factor list.

For reversal, reverse all edges retaining labels. A fresh start epsilon-branches to every old accepting state; only the old initial state accepts. Reverse each finite old path to reverse its symbol word, proving both language inclusions. If the old accepting set is empty, the new machine has no entry to old states and still accepts nothing.

Together with determinization, these prove regular-language closure under union, concatenation, star, and reversal. Total-DFA constructions prove intersection, difference, and complement.

Regular-expression syntax and meaning

Regular expressions start with $\varnothing, \varepsilon, a$ for $a \in \Sigma$, and finitely apply union $R|S$, concatenation RS, and star R^*. Their languages are respectively empty, singleton empty word, singleton symbol, and the associated language operations.

Star binds before concatenation, which binds before union. Expression $(0|1)^*10$ describes arbitrary binary prefixes followed by 10, matching the preceding NFA. Changing parentheses to $0|1^*10$ changes the language.

Each expression yields an NFA. The three base cases use no accepting path, one epsilon path, or one symbol edge. The union, concatenation, and star constructions handle its syntax tree inductively. This proves existence and gives a terminating algorithm, since a finite tree invokes finitely many finite-state constructions.

A path recurrence from automata to expressions

Number NFA states one through n. Let $R_{ij}^{(k)}$ denote paths from i to j whose internal states are among one through k. Base $R_{ij}^{(0)}$ unions direct-edge labels, also adding ε for i=j, and uses $\varnothing$ if no edge exists between distinct states.

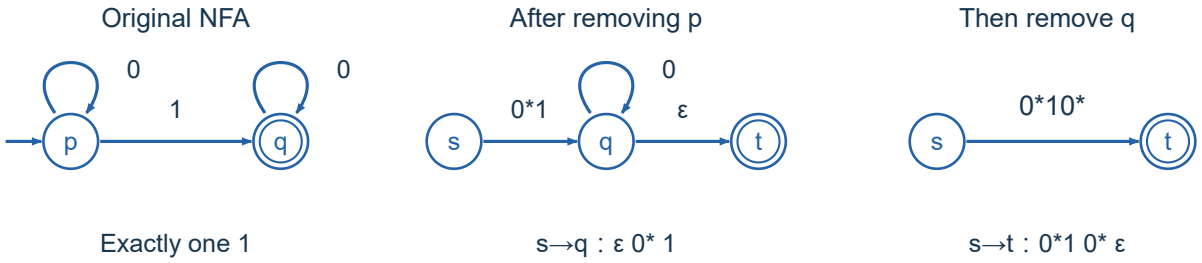
$$R_{ij}^{(k)} = R_{ij}^{(k-1)} \mid R_{ik}^{(k-1)} (R_{kk}^{(k-1)})^{*} R_{kj}^{(k-1)}.$$

Paths without internal k give the first term. Otherwise split at successive visits to k into a first i-to-k segment, zero or more k-to-k segments, and a final k-to-j segment, each internally avoiding k. Conversely concatenating these segments uses only permitted internal states. Empty first or last segments cover endpoints equal to k.

Union the final $R^{(n)}$ from start to each accepting state to obtain the language expression. Round k must use previous-round values rather than overwritten current values. Finitely many rounds terminate, but expanded expressions may be large; no size-preservation claim follows.

A concrete state-elimination example

State elimination: incoming edge × loop star × outgoing edge



Star permits zero repetitions, so word 1 must also accept.

The start state p and accepting state q each have a 0-loop; p has a 1-edge to q and there are no other edges. Add a new start s with an ε-edge to p, and an ε-edge from q to a new accepting state t.

State elimination: update each label

Eliminating p creates the s-to-q label $\varepsilon 0^* 1 = 0^* 1$. Eliminating q then gives the s-to-t label $(0^*1)0^*\varepsilon = 0^*10^*$. These words contain exactly one 1, with any number of zeros before and after.

If another s-to-q edge already existed, take the union with the new label; do not overwrite it. The loop star permits zero repetitions; omitting that case would incorrectly reject the word 1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Equivalent states agree on every future suffix

Define $p \equiv q$ when every $w \in \Sigma^*$ has the same acceptance result starting at p and q. This is an equivalence relation; the empty suffix forces equal acceptance marks.

For any symbol a and suffix w, the concatenation lemma rewrites acceptance after $\delta(p, a)$ on w as acceptance after p on aw. Thus $p \equiv q$ implies $\delta(p, a) \equiv \delta(q, a)$. Quotient transitions $[p] \xrightarrow{a} [\delta(p, a)]$ are independent of representatives, as are the initial and accepting classes.

Induction on input length makes the quotient's final class equal the class of the original final state, preserving the language. Nearby drawings or identical one-step labels cannot replace the universal-suffix condition. Chapter 7 computes this partition effectively.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Equivalence and shortest distinguishing words

For two total DFAs on one alphabet, construct a product accepting when exactly one component accepts. Its language is their symmetric difference. The machines are equivalent iff no accepting product state is reachable from the start.

BFS of reachable pairs finds a shortest distinguishing word by backtracking labels. If the initial pair already accepts, the word is empty. Otherwise a shortest path repeats no state and has length at most $|Q_1||Q_2| - 1$. Exhausting reachable pairs without acceptance proves equal results for all words via the projection invariant.

Switching the even-ones machine's accepting state from E to O makes their initial results disagree, so empty input is the shortest counterexample. Exhaustive enumeration of all words below the bound is unnecessary.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pumping lemma and its quantifiers

For a p-state DFA, every accepted word of length at least p has a split $w = xyz$, with $|xy| \leq p$, $|y| \geq 1$, such that xy^tz is accepted for every $t \geq 0$.

The first p input symbols pass through p+1 state positions, so positions $i < j \leq p$ repeat a state. Let x end at i, y contain the next j−i symbols, and z the remainder. The nonempty y labels a loop, traversable zero or arbitrarily many times before the same z, preserving acceptance.

A nonregularity proof must defeat every permitted split, choosing a suitable repetition count, rather than one convenient split. Pumpability is a necessary condition, not a sufficient machine construction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Equal blocks of zeros and ones need unbounded memory

For $L = \{0^n 1^n : n \geq 0\}$, suppose a p-state DFA exists and choose $0^p 1^p$. Every pumping split has a nonempty y entirely inside the initial zeros. Taking t=0 removes zeros but leaves p ones, producing a word outside L, contradiction.

Independently, among $\varepsilon, 0, \ldots, 0^p$, two prefixes $0^i, 0^j$, i<j, reach one state. Append 1^i. The first word belongs to L and the second does not, contradicting identical future behavior.

The impossibility concerns a fixed finite-state machine with unbounded input lengths. Bounding the number of zeros in advance allows finitely many counting states and does not conflict with the result.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Recursive balanced parentheses are not regular

Prefixes with arbitrarily many opening parentheses require arbitrarily many distinct histories. If prefixes with i<j openings reached the same state, appending i closings would balance the first but leave j−i unmatched openings in the second, contradiction. Pigeonhole guarantees such a collision for every proposed finite state count, so the language is not regular.

Its earlier recursive definition remains valid. Recursive generation is different from finite-state memory. A growing stack or counter can retain unbounded unmatched depth; a fixed maximum depth permits a finite-state encoding. The boundary depends on distinguishable histories, not visual graph complexity.

References and attribution

Lin Cing, handwritten discrete mathematics notes. Ralph P. Grimaldi, Discrete and Combinatorial Mathematics, 5th ed., Ch. 6.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study